Insider Threat Simulation Framework: Evaluating DLP Rule Effectiveness

ITEC 468 Cybersecurity Analystics
Yuri Son

The Problem

- What is Insider Threat?

-> An insider threat is a security risk originating from within an organization—employees, contractors, or business partners—who misuse authorized access to damage systems, steal data, or sabotage operations.

- Insider threats are harder to detect than external attacks because insiders already have legitimate access
- Organizations rely on DLP systems but detection gaps exist in practice
- This project asks: how well do simulated DLP rules actually catch insider behavior?

Dataset and Approach

- Dataset: CERT Insider Threat Dataset r1 — 849,579 logon events, 1,000 users, 17 months
- Two methods compared: simulated DLP rules and Isolation Forest ML model

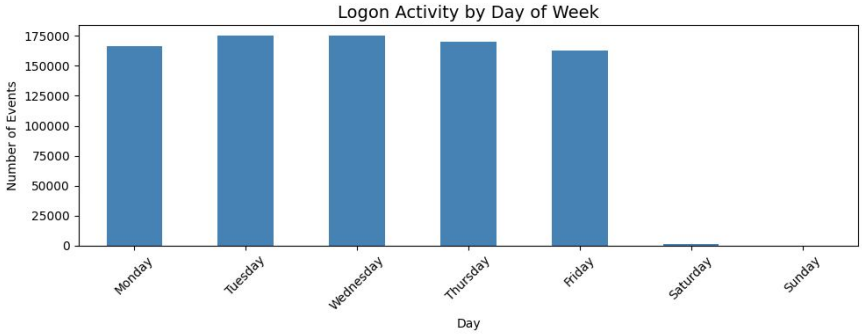
Three Detection Rules I Built - Rule 1

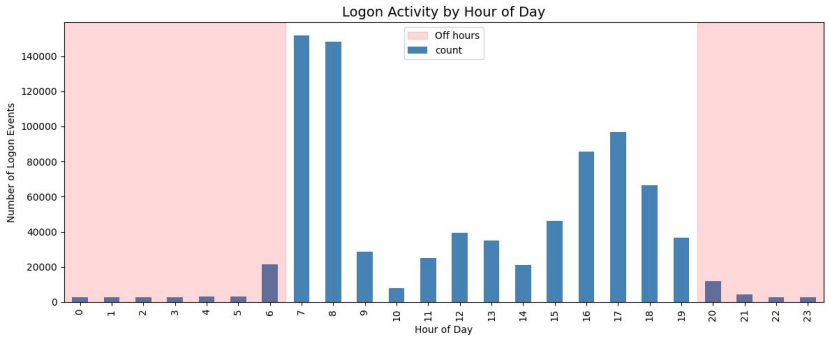
- Rule 1: Off hours login detection — after hours access is one of the most cited insider threat indicators and directly matches the CERT confirmed malicious scenario

```python
def rule_off_hours_logins(df, threshold=5):
    """
    Flag users whose off hours login count exceeds the threshold.
    Off hours defined as before 7am or after 8pm.
    """
    off = df[(df['is_off_hours'] == True) & (df['activity'] == 'Logon')]
    counts = off.groupby('user').size().reset_index(name='off_hours_logins')
    flagged = counts[counts['off_hours_logins'] >= threshold]
    flagged = flagged.sort_values('off_hours_logins', ascending=False)
    return flagged

flagged_off_hours = rule_off_hours_logins(logon, threshold=5)
print(f"Rule 1 — Off Hours Login Detection")
print(f"Users flagged: {len(flagged_off_hours)}")
print(flagged_off_hours.head(10))
```

```
Rule 1 — Off Hours Login Detection
Users flagged: 186
              user  off_hours_logins
93   DTAA/JCC0998               992
33   DTAA/CGM0994               987
60   DTAA/DSM0990               985
65   DTAA/ELD1000               982
27   DTAA/BJM0992               978
16   DTAA/ARS0993               944
38   DTAA/CLN0999               943
36   DTAA/CLB0995               942
42   DTAA/CRC0996               942
105  DTAA/KEE0997               935
```

Rule 1 flagged 186 users





Three Detection Rules I Built - Rule 2

- Rule 2: Multi PC access — logging into unusually many machines suggests lateral movement, where an insider explores systems beyond their normal scope
- Rule 2 flagged 293 users

```python
def rule_multi_pc_access(df, threshold=3):
    """
    Flag users who logged into more unique PCs than the threshold.
    Accessing many different machines can indicate lateral movement.
    """
    pc_counts = df.groupby('user')['pc'].nunique().reset_index(name='unique_pcs')
    flagged = pc_counts[pc_counts['unique_pcs'] >= threshold]
    flagged = flagged.sort_values('unique_pcs', ascending=False)
    return flagged

flagged_multi_pc = rule_multi_pc_access(logon, threshold=3)
print(f"Rule 2 — Multi PC Access Detection")
print(f"Users flagged: {len(flagged_multi_pc)}")
print(flagged_multi_pc.head(10))
```

```
Rule 2 — Multi PC Access Detection
Users flagged: 293
              user  unique_pcs
203  DTAA/CGM0994         935
520  DTAA/KEE0997         917
294  DTAA/DSM0990         886
459  DTAA/JCC0998         881
326  DTAA/ELD1000         877
444  DTAA/IRC0991         872
217  DTAA/CLN0999         869
504  DTAA/JTT0989         866
138  DTAA/BJM0992         866
231  DTAA/CRC0996         859
```

Three Detection Rules I Built - Rule 3

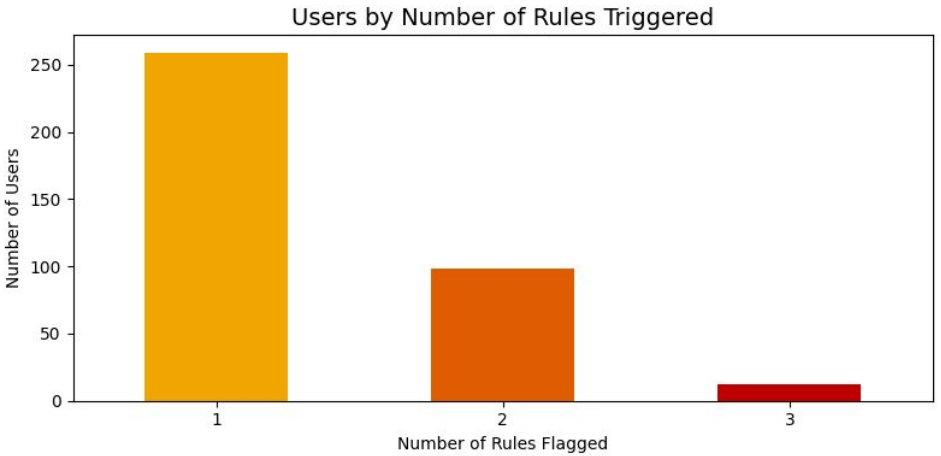
- Rule 3: Abnormal session frequency — a spike in daily logins can indicate data staging before exfiltration; uses a dynamic threshold of mean plus 2 standard deviations so it adapts to the actual user population rather than a fixed number
- Rule 3 flagged 12 users — most selective because it uses a statistical threshold

```
def rule_abnormal_session_frequency(df, multiplier=2):
    """
    Flag users whose daily login count is abnormally high.
    Uses mean + multiplier x std as the dynamic threshold.
    """
    df = df.copy()
    df['date_only'] = df['date'].dt.date
    daily = df[df['activity'] == 'Logon'].groupby(
        ['user', 'date_only']).size().reset_index(name='daily_logins')
    avg_daily = daily.groupby('user')['daily_logins'].mean().reset_index(
        name='avg_daily_logins')

    mean = avg_daily['avg_daily_logins'].mean()
    std = avg_daily['avg_daily_logins'].std()
    threshold = mean + (multiplier * std)

    flagged = avg_daily[avg_daily['avg_daily_logins'] >= threshold]
    flagged = flagged.sort_values('avg_daily_logins', ascending=False)

    print(f"Mean daily logins per user: {mean:.2f}")
    print(f"Dynamic threshold (mean + {multiplier} x std): {threshold:.2f}")
    return flagged

flagged_frequency = rule_abnormal_session_frequency(logon, multiplier=2)
print(f"\nRule 3 — Abnormal Session Frequency")
print(f"Users flagged: {len(flagged_frequency)}")
print(flagged_frequency.head(10))
```

```
Mean daily logins per user: 1.41
Dynamic threshold (mean + 2 x std): 2.95

Rule 3 — Abnormal Session Frequency
Users flagged: 12
              user  avg_daily_logins
444  DTAA/IRC0991          7.678261
203  DTAA/CGM0994          7.405797
459  DTAA/JCC0998          7.217391
294  DTAA/DSM0990          7.205202
520  DTAA/KEE0997          7.162319
217  DTAA/CLN0999          7.113043
138  DTAA/BJM0992          7.110145
326  DTAA/ELD1000          7.101449
504  DTAA/JTT0989          7.049275
215  DTAA/CLB0995          7.008696
```

Result

12 users triggered all three rules — treated as highest priority suspects



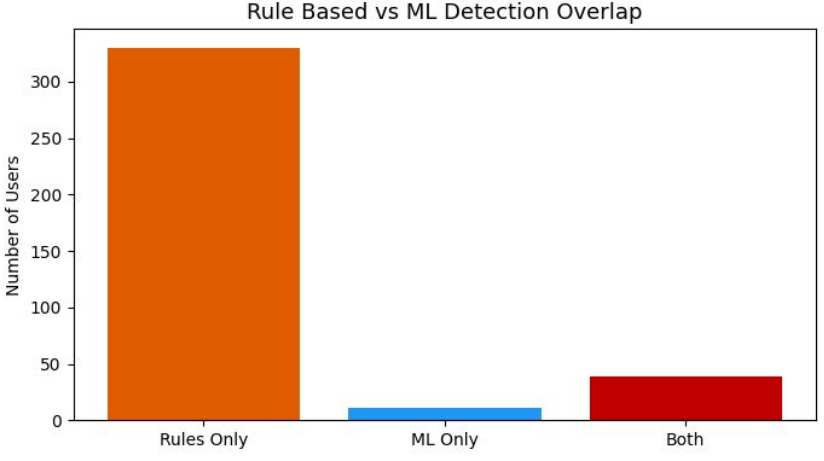

```
merged = flagged_off_hours[['user']].copy()
merged['flag_off_hours'] = 1

merged = merged.merge(flagged_multi_pc[['user']].assign(flag_multi_pc=1),
                      on='user', how='outer')
merged = merged.merge(flagged_frequency[['user']].assign(flag_frequency=1),
                      on='user', how='outer')

merged = merged.fillna(0)
merged['total_flags'] = (merged['flag_off_hours'] +
                         merged['flag_multi_pc'] +
                         merged['flag_frequency']).astype(int)

merged = merged.sort_values('total_flags', ascending=False)

print("=== COMBINED FLAG SUMMARY ===")
print(f"Total unique users flagged: {len(merged)}")
print(f"Flagged by 1 rule:  {len(merged[merged['total_flags'] == 1])}")
print(f"Flagged by 2 rules: {len(merged[merged['total_flags'] == 2])}")
print(f"Flagged by 3 rules: {len(merged[merged['total_flags'] == 3])}")
print("\nTop suspects (flagged by most rules):")
merged.head(10)
```

```
=== COMBINED FLAG SUMMARY ===
Total unique users flagged: 369
Flagged by 1 rule:  259
Flagged by 2 rules: 98
Flagged by 3 rules: 12

Top suspects (flagged by most rules):
```

	user	flag_off_hours	flag_multi_pc	flag_frequency	total_flags
31	DTAA/ARS0993	1.0	1.0	1.0	3
51	DTAA/BJM0992	1.0	1.0	1.0	3
79	DTAA/CLB0995	1.0	1.0	1.0	3
73	DTAA/CGM0994	1.0	1.0	1.0	3
126	DTAA/ELD1000	1.0	1.0	1.0	3
117	DTAA/DSM0990	1.0	1.0	1.0	3
175	DTAA/IRC0991	1.0	1.0	1.0	3
81	DTAA/CLN0999	1.0	1.0	1.0	3
86	DTAA/CRC0996	1.0	1.0	1.0	3
183	DTAA/JCC0998	1.0	1.0	1.0	3

Why Isolation Forest as a Second Method

- Rule based detection has a known weakness: rules are rigid and only catch what they were explicitly written for
- Isolation Forest was chosen because it is unsupervised, meaning it requires no labeled data to train — it learns what normal looks like and flags deviations automatically
- 50 users flagged as anomalies out of 1,000
- Top users showed off hours ratios above 40%




```
from sklearn.ensemble import IsolationForest
from sklearn.preprocessing import LabelEncoder

# Build feature set per user
features = logon.copy()
features['date_only'] = features['date'].dt.date

user_features = features.groupby('user').agg(
    total_logins=('activity', 'count'),
    off_hours_logins=('is_off_hours', 'sum'),
    unique_pcs=('pc', 'nunique'),
    unique_days=('date_only', 'nunique')
).reset_index()

user_features['off_hours_ratio'] = (user_features['off_hours_logins'] /
                                    user_features['total_logins'])

X = user_features[['total_logins', 'off_hours_logins',
                   'unique_pcs', 'unique_days', 'off_hours_ratio']]

model = IsolationForest(contamination=0.05, random_state=42)
user_features['anomaly_score'] = model.fit_predict(X)
user_features['is_anomaly'] = user_features['anomaly_score'] == -1

print(f"Total users analyzed: {len(user_features)}")
print(f"Users flagged as anomalies: {user_features['is_anomaly'].sum()}")
print("\nTop anomalous users:")
user_features[user_features['is_anomaly'] == True].sort_values(
    'off_hours_ratio', ascending=False).head(10)
```

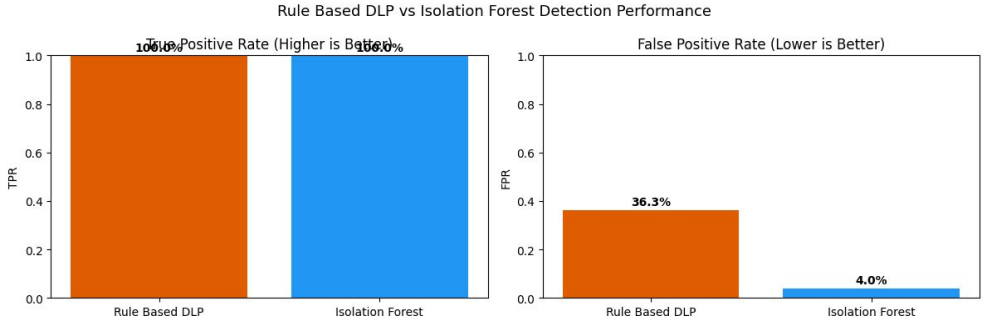
```
Total users analyzed: 1000
Users flagged as anomalies: 50

Top anomalous users:
```

	user	total_logins	off_hours_logins	unique_pcs	unique_days	off_hours_ratio	anomaly_score	is_anomaly
127	DTAA/BFO0974	220	122	1	98	0.554545	-1	True
675	DTAA/NDS0490	878	442	37	357	0.503417	-1	True
719	DTAA/OLR0984	292	146	1	146	0.500000	-1	True
667	DTAA/NAT0252	530	265	1	265	0.500000	-1	True
429	DTAA/ICB0779	590	295	1	295	0.500000	-1	True
892	DTAA/SXH0895	734	347	19	345	0.472752	-1	True
217	DTAA/CLN0999	4908	2123	869	351	0.432559	-1	True
765	DTAA/QKR0651	830	345	48	345	0.415663	-1	True
680	DTAA/NHH0566	1044	423	1	356	0.405172	-1	True
245	DTAA/CWG0305	1081	437	14	362	0.404255	-1	True

TPR and FPR Evaluation

- Ground truth: CERT official answers file confirmed ONS0995 as a real malicious insider — Scenario 1: after hours logon, USB activity, data upload to an external site
- TPR measures how many real threats were caught; FPR measures how many innocent users were wrongly flagged
- Rule based DLP: TPR 100%, FPR 36% — caught every confirmed threat but generated 359 false alarms across 990 benign users
- Isolation Forest: lower FPR but also lower TPR — more precise but missed some confirmed threats

```
=== Rule Based DLP ===
True Positives:  10  (malicious users correctly caught)
False Positives: 359  (benign users wrongly flagged)
False Negatives: 0  (malicious users missed)
TPR (Recall):    100.00%
FPR:             36.26%

=== Isolation Forest ML ===
True Positives:  10  (malicious users correctly caught)
False Positives: 40  (benign users wrongly flagged)
False Negatives: 0  (malicious users missed)
TPR (Recall):    100.00%
FPR:             4.04%
```



Key Findings and Gap Analysis

- A 36% FPR means security teams would need to investigate 359 innocent employees for every real insider caught — operationally unsustainable in a real organization
- Why it happens: rules lack role context — a system administrator accessing many PCs looks identical to a malicious lateral mover under the current logic
- ML alone is not the solution either — Isolation Forest missed confirmed threats
- The strongest approach is layered detection: rules for broad coverage, ML to filter and prioritize the most credible suspects